Extended Spec Language Design

Toward Editor-Assisted Interactive Development

Prologos Project — from Zachary Larson

2026-02-22

Contents

1 Executive Summary **5**
1.1 Keyword Symmetry Table 5

2 Part I: Research Survey **6**
2.1 1. Clojure Spec & Malli 6
2.1.1 Clojure Spec 6
2.1.2 Malli 6
2.1.3 Insight for Prologos 7
2.2 2. PropEr (Erlang Property-Based Testing) 7
2.3 3. Agda/Idris Interactive Development 8
2.3.1 Agda 8
2.3.2 Idris 2 8
2.3.3 Insight for Prologos 9
2.4 4. Dependent Type Ergonomics 9
2.4.1 How Languages Present Pi Types 9
2.4.2 Lean's `abbrev` vs `def` Distinction 9
2.4.3 Type-Level Functions in Dependently-Typed Languages 10
2.4.4 Insight for Prologos 10
2.5 5. Refinement Types (Liquid Haskell, F*) 10
2.5.1 Liquid Haskell 10
2.5.2 F* 10
2.5.3 Insight for Prologos 11
2.6 6. Contract Systems & Spec Language Design Patterns 11
2.6.1 Racket Contracts 11
2.6.2 Composability Patterns 11
2.6.3 Koka Effect Aliases 11

2.6.4 Insight for Prologos . . . 11
2.7 7. Type Abbreviation Mechanisms (Cross-Language Survey) . 12
2.7.1 Transparent Aliases . . . 12
2.7.2 Opaque / Controlled-Reducibility Aliases . . . 12
2.7.3 Module-Level Abstraction . . . 12
2.7.4 Insight for Prologos . . . 12

3 Part II: Syntax Design 12
3.1 Current Spec Syntax . . . 12
3.2 Extended Spec Syntax . . . 13
3.2.1 The `:implicits` Key . . . 13
3.2.2 Multi-Arity Specs with Metadata . . . 14
3.2.3 Sexp Mode Equivalent . . . 14
3.2.4 How Parsing Works . . . 15
3.3 Supported Keys . . . 15
3.3.1 Phase 1 (Near-term: syntax + storage) . . . 15
3.3.2 Phase 2 (Medium-term: property checking) . . . 15
3.3.3 Phase 3 (Long-term: verification) . . . 16
3.4 Mapping Spec Keys to Dependent Types . . . 16
3.4.1 Properties as Dependent Types . . . 16
3.4.2 Preconditions as Refined Arguments . . . 16
3.4.3 Postconditions as Refined Return Types . . . 16
3.4.4 Invariants as Dependent Function Types . . . 17
3.4.5 The Graduation Path . . . 17

4 Part III: Typed Holes (`??`) 17
4.1 Motivation . . . 17
4.2 Syntax . . . 18
4.2.1 Reader . . . 18
4.2.2 Distinction from `_` . . . 18
4.2.3 Hole Report . . . 18
4.3 "The What?What?" Interaction Model . . . 19

5 Part IV: The `property` Keyword 19
5.1 Motivation . . . 19
5.2 Syntax . . . 20
5.2.1 Basic Declaration . . . 20
5.2.2 Usage in `spec` . . . 20
5.3 Composition via `:includes` . . . 20
5.4 The Logic Programming Reading . . . 21

5.5 Parameterized Properties (Higher-Order) 21
5.6 `:laws` on Traits 22
5.7 Hierarchical Laws (Functor → Applicative → Monad) 23
5.8 Name Scoping in `:includes` 23
5.9 The Symmetry 24

6 Part V: Expressing Higher-Kinded Pi Types Ergonomically 24
6.1 The Problem 24
6.2 Progressive Complexity Levels 24
6.2.1 Level 0: No types (inference only) 24
6.2.2 Level 1: Simple spec (monomorphic) 24
6.2.3 Level 2: Polymorphic spec 25
6.2.4 Level 3: Constrained polymorphism 25
6.2.5 Level 4: Higher-kinded polymorphism 25
6.2.6 Level 5: Named type abstractions (functor) 25
6.2.7 Level 6: Dependent types (Pi) 25
6.2.8 Level 7: Dependent types (Sigma / existential) 26
6.3 Key Ergonomic Decisions 26

7 Part VI: Improved Implicit Inference 26
7.1 Direction 1: Auto-introduce unbound type variables 27
7.2 Direction 2: Kind inference from `:where` clauses 27
7.3 Explicit Always Available 27

8 Part VII: The `functor` Keyword 27
8.1 Motivation 27
8.2 Syntax 28
8.3 Progressive Disclosure 28
8.4 Category-Theoretic Metadata Keys 29
8.5 Why `functor` and Not `abbrev` or `type` 29
8.6 Transparency Model 30
8.7 Error Messages 30
8.8 How `functor` Composes with `spec`, `property`, and `trait` . . 30

9 Part VIII: Implementation Considerations 31
9.1 spec-entry Struct Extension 31
9.2 rewrite-implicit-map Extension 32
9.3 Typed Hole AST Node 32
9.4 property-entry Store 32
9.5 functor-entry Store 33

9.6 `:implicits` Merging in process-spec 33
9.7 Grammar Extensions 33

10 Part IX: Worked Examples **34**
10.1 Example 1: Simple spec with documentation, examples, and reusable properties 34
10.2 Example 2: Contract-style spec 35
10.3 Example 3: Higher-kinded functor laws (using property + :laws) 35
10.4 Example 4: Transducer module (functor + property + spec) . 36
10.5 Example 5: Interactive development with ?? 37
10.6 Example 6: Algebraic property hierarchy 37
10.7 Example 7: Optics (functor for Lens) 38
10.8 Example 8: Refinement type (future) 38
10.9 Example 9: Dependent-type-heavy spec 39

11 Part X: Phased Roadmap **39**
11.1 Phase 1: Syntax and Storage (Near-term) 39
11.2 Phase 1b: Improved Implicit Inference 40
11.3 Phase 2: Example and Property Checking (Medium-term) . . 40
11.4 Phase 3: Refinement and Verification (Long-term) 40
11.5 Phase 4: Interactive Theorem Proving (Far future) 40

12 Part XI: Resolved Design Questions **41**

13 References **41**
13.1 Clojure Spec 41
13.2 Malli 42
13.3 PropEr 42
13.4 Agda 42
13.5 Idris 2 42
13.6 Lean 4 42
13.7 Liquid Haskell 42
13.8 Haskell Type Synonyms 42
13.9 Koka 42
13.10 F* 42
13.11 1ML 43
13.12 Racket Contracts 43

1 Executive Summary

This document proposes extending Prologos's surface language with three coordinated features that make dependent types, properties, and higher-kinded abstractions approachable without sacrificing formal rigor:

1. **Extended** `spec` with keyword metadata (`:implicits`, `:where`, `:doc`, `:examples`, `:properties`, contracts, refinements)

2. `property` **keyword** for reusable, composable proposition groups (analogous to `bundle` for traits), with `:includes` composition and `:laws` on traits

3. `functor` **keyword** for named type abstractions with category-theoretic metadata (`:unfolds`, `:compose`, `:identity`, `:laws`) — eliminating raw Pi/Sigma types from the surface language

4. `??` **typed holes** for interactive, hole-driven development

The design philosophy: *dependent types are the implementation substrate; the surface language speaks in domain terms.* Pi types, Sigma types, universe levels, and multiplicity annotations exist in `:unfolds` blocks, in the elaborator, and in the type checker. The primary programmer interface is structured metadata expressed through keywords.

The design is informed by a survey of Clojure Spec, Malli, PropEr, Agda, Idris, Lean 4, Haskell, F*, Koka, and Racket contracts. Property-based testing and interactive theorem proving are explicit design *targets* (we design syntax to support them) but not implementation targets (we do not build the engines yet).

1.1 Keyword Symmetry Table

Abstraction layer	Keyword	What it names	Analogous to
Function interface	`spec`	Function type + metadata	ML type signatures
Method requirement	`trait`	A single method signature	Haskell typeclass meth
Requirement group	`bundle`	Conjunction of traits	Haskell superclass con
Proposition group	`property`	Conjunction of `:holds` clauses	QuickCheck property s
Type abstraction	`functor`	Parameterized type + algebraic structure	CT functor / Lean `abb`

2 Part I: Research Survey

2.1 1. Clojure Spec & Malli

2.1.1 Clojure Spec

Clojure Spec (`clojure.spec.alpha`) is *predicate-first*: every spec is fundamentally a predicate function. The key insight for Prologos is the three facets of `s/fdef`:

```
(s/fdef ranged-rand
  :args (s/cat :start int? :end int?)
  :ret  int?
  :fn   (fn [{:keys [args ret]}]
          (and (>= ret (:start args))
               (<  ret (:end args)))))
```

- `:args`: regex spec for function arguments (`s/cat`, `s/alt`)
- `:ret`: spec for the return value
- `:fn`: predicate relating `{:args conformed-args :ret conformed-ret}`

The `:fn` relation is the most interesting for Prologos — it expresses *properties* that must hold between inputs and outputs. In a dependently-typed language, this corresponds directly to a dependent Pi type.

Generative testing: `stest/check` generates random inputs from `:args`, invokes the function, and checks both `:ret` and `:fn`. Specs *are* generators — no separate test-writing step required. This is QuickCheck-style property testing with zero user-written generators.

Instrument vs check: `instrument` validates `:args` at call sites (development-time contract enforcement). `check` validates `:ret` and `:fn` via generative testing (test-time property verification). This separation is important: *contracts at dev time, properties at test time.*

Composition: `s/keys` for entity maps, `s/cat` for sequential concatenation, `s/alt` for alternatives, `s/and` for conjunctive refinement. Specs compose algebraically.

2.1.2 Malli

Malli is *schema-first* (data-driven): schemas are plain data structures, not opaque predicate objects. This enables schema transformation, generation, serialization.

```
(def MyFunction
  [:=> [:cat :int :int] :int])

;; Schema-as-data enables programmatic manipulation:
(m/children MyFunction) ;; => [[:cat :int :int] :int]
```

Key lesson: In a homoiconic language like Prologos, spec metadata should be data — quotable, inspectable, transformable. This is naturally satisfied by using maps.

2.1.3 Insight for Prologos

In a dependently-typed language, types already subsume specs. Clojure's `:fn` relation corresponds to a dependent Pi type:

```
;; Clojure spec's :fn relation for ranged-rand:
;; (fn [{:keys [args ret]}] (and (>= ret start) (< ret end)))

;; Is expressible as the dependent type:
;; <(start : Int) -> (end : Int) -> {v : Int | v >= start, v < end}>
```

Runtime property testing remains valuable even when types encode the invariant: the type checker catches *logical* errors at compile time; property testing catches *implementation* bugs.

2.2 2. PropEr (Erlang Property-Based Testing)

PropEr is notable for its *type-driven generator synthesis*. If a function has a Dialyzer type spec, PropEr constructs generators automatically:

```
-spec foo(integer(), string()) -> boolean().
%% PropEr generates random (integer, string) pairs and checks boolean result
```

Compositional shrinking: When custom generators are built from other generators, shrinking behavior inherits automatically. Data structures containing other data "tend to empty themselves."

Property patterns: `?FORALL(Xs, Generator, Property)` is the canonical form. `?IMPLIES` for conditional properties. `aggregate/collect` for coverage.

Insight for Prologos: Our types are richer than Dialyzer specs. A `spec` with dependent types gives enough information to auto-generate random inputs. The generator infrastructure should be type-directed: `Gen : Type -> Generator` as a trait, with automatic derivation for algebraic data types.

2.3 3. Agda/Idris Interactive Development

2.3.1 Agda

Agda's hole-driven development is the gold standard for interactive type-driven programming:

- **Holes**: `?` in code creates a *hole* (placeholder for incomplete code). The type checker reports the expected type and available local bindings.
- **Commands**:

Key	Action	Description
`C-c C-r`	Refine	Fill hole with expression + new sub-holes
`C-c C-c`	Case	Split on a variable (pattern match)
`C-c C-a`	Auto	Proof search (try constructors)
`C-c C-,`	Goal	Show expected type and context

- **Auto proof search**: Completely rewritten in Agda 2.7.0. Searches for type inhabitants by trying constructors and projections. Works for "small enough problems of almost any kind." Users provide hints via constants or scope.

2.3.2 Idris 2

Idris's interactive editing provides similar capabilities with a different flavor:

- **Holes**: `?name` syntax. The REPL shows expected type and context.
- **Commands**:

Command	Description
`:ac`	Add clause template for a function
`:cs`	Case split on variable (removes impossible cases)
`:am`	Add missing clauses
`:ps`	Proof search
`:mw`	Make with clause

- **Case splitting** (`:cs`): Removes *impossible* cases due to unification — this is type-directed case analysis. When a constructor can't unify with the expected type, the case is silently omitted.

2.3.3 Insight for Prologos

The `??` syntax naturally maps to our existing `expr-hole` infrastructure (`syntax.rkt` line 688). The type checker already handles `expr-hole` — it accepts any type. The missing piece is the *interactive protocol*: the editor asks "what type is expected here, what bindings are in scope?" and the type checker reports back.

Key distinction: `_` means "infer this for me automatically" (silent, automatic). `??` means "I don't know what goes here, please help me" (interactive, informational). The former is for the machine; the latter is for the human.

2.4 4. Dependent Type Ergonomics

2.4.1 How Languages Present Pi Types

Language	Pi syntax	Implicit	Instance
Agda	`(x : A) -> B`	`{x : A}`	`{{x : A}}`
Idris 2	`(x : A) -> B`	`{x : A}`	`{auto x : A}`
Lean 4	`(x : A) -> B`	`{x : A}`	`[x : A]`
Prologos	`<(x : A) -> B>`	`{A : Type}`	where clause

The ergonomic tension: Pi types with many parameters become hard to read. All dependently-typed languages solve this the same way: separate the *signature* from the *definition*. Prologos's `spec~/~defn` split already achieves this. The extended spec pushes further by structuring the metadata around the signature.

2.4.2 Lean's `abbrev` vs `def` Distinction

Lean 4 distinguishes transparent abbreviations from opaque definitions:

- `abbrev` is tagged `@[reducible]` — unfolded during elaboration, instance synthesis, and definitional equality checks. `abbrev MyMonad := ReaderT Env IO` means `Monad MyMonad` resolves automatically via `Monad (ReaderT ...)`.
- `def` (semireducible) is *not* unfolded by instance synthesis. You need explicit instances for opaque type definitions.

This distinction is critical for type aliases that should "carry through" trait instances vs abstract types that deliberately hide their representation.

2.4.3 Type-Level Functions in Dependently-Typed Languages

In Agda, Idris, and Lean, type synonyms are ordinary functions returning `Type`. No special mechanism needed. The tradeoff: full type-level computation makes inference harder and error messages may show fully-reduced forms rather than the user's abbreviation name.

Haskell's `type` synonyms are transparent but *cannot be partially applied* — a fundamental restriction. Prologos should avoid this limitation.

2.4.4 Insight for Prologos

Our existing implicit argument inference (Sprint 3) and trait resolution (Phases A–E) provide the core mechanisms. The ergonomic challenge is *presentation* — how to show complex Pi types without overwhelming users. The extended `spec` addresses function-level presentation; the `functor` keyword (Part VII) addresses type-level presentation.

2.5 5. Refinement Types (Liquid Haskell, F*)

2.5.1 Liquid Haskell

Refinement types refine base types with logical predicates:

```
{v : T | predicate}
```

For functions, the return type can reference the input:

```
incr :: x:Int -> {v:Int | v = x + 1}
```

Type checking reduces to SMT validity queries (static verification, not runtime testing). Refinement reflection allows the SMT solver to reason about recursive function implementations.

2.5.2 F*

Combines ML-like programming with dependent types, monadic effects, and refinement types. Uses Z3 for automated verification. Programs extract to OCaml, F#, or C.

F* provides fine-grained unfolding control: `unfold` (always reduce), `unfold_for_unification_and_v` (reduce only during unification and VC generation), or opaque (default). This three-tier system gives precise control over where abbreviations are transparent.

2.5.3 Insight for Prologos

Prologos already has full dependent types, which *subsume* refinement types: `{v : Int | v > 0}` in Liquid Haskell is expressible as `<(v : Int) * (GT v 0)>` (Sigma type pairing value with proof) in Prologos.

The ergonomic question: should we offer a lighter-weight refinement syntax (`:refines` key) that *compiles* to Sigma types? Yes — for pragmatism. Users write `:refines (fn [r] [>= r 0])` and the compiler generates the Sigma type.

2.6 6. Contract Systems & Spec Language Design Patterns

2.6.1 Racket Contracts

`->i` for dependent contracts with `#:pre` and `#:post` conditions. `define/contract` establishes a contract boundary. Blame tracking identifies which party violated the contract.

2.6.2 Composability Patterns

- The Specification Pattern (Fowler) composes predicates via AND/OR/NOT combinators.
- The "schema explosion" problem occurs when specs grow combinatorially. Solution: keep specs *open* (new keys without modifying existing specs) and *compositional* (specs combine via simple operations). Clojure handles this via `s/merge` and `s/and`.

2.6.3 Koka Effect Aliases

Koka's `alias pure = <div,exn>` abbreviates a row of effects. This demonstrates the general principle: named groupings of constraints improve readability. Prologos's `bundle` already does this for trait constraints; `property` will do this for propositions; `functor` will do this for type structure.

2.6.4 Insight for Prologos

The map argument to `spec` is inherently open and composable. New keys can be added in future versions without breaking existing code. Unrecognized keys are stored but not acted upon (*Postel principle*: liberal in what you accept, conservative in what you emit). Spec metadata should be first-class data (homoiconicity principle) — quotable, inspectable, transformable.

2.7 7. Type Abbreviation Mechanisms (Cross-Language Survey)

2.7.1 Transparent Aliases

Haskell `type`, Rust `type`, Koka `alias`. Fully expanded before type checking; zero impact on inference. Haskell's limitation: type synonyms cannot be partially applied. Rust and Koka do not share this restriction.

2.7.2 Opaque / Controlled-Reducibility Aliases

Lean 4's `abbrev` (reducible, instances carry through) vs `def` (semireducible, opaque to instance search). Scala 3's `opaque type` (transparent inside defining scope, opaque outside). Haskell's `newtype` (distinct type, zero-cost at runtime).

2.7.3 Module-Level Abstraction

OCaml modules control type visibility through signatures: same `type t = int` can be exposed transparently or abstractly depending on the `.mli` file. 1ML (Rossberg) unifies core and module language so type abbreviations are ordinary first-class functions.

2.7.4 Insight for Prologos

The `functor` keyword (Part VII) takes the Lean `abbrev` approach — transparent by default, with the added richness of category-theoretic metadata. Unlike bare abbreviations, a `functor` declares what the type *means* through its operations, not just what it expands to.

3 Part II: Syntax Design

3.1 Current Spec Syntax

```
;; Basic:
spec add Nat Nat -> Nat

;; With implicit binders:
spec id {A : Type} A -> A

;; With trait constraints (where clause):
spec sort {A : Type} [List A] -> [List A] where (Ord A)
```

```
;; With inline constraints:
spec elem {A} [Eq A] A [List A] -> Bool

;; Multi-arity:
spec zip
  | {A B : Type} [List A] [List B] -> [List [Pair A B]]
  | {A B C : Type} [A -> B -> C] [List A] [List B] -> [List C]
```

The `spec-entry` struct (macros.rkt line 203) stores: `type-datums`, `docstring`, `multi?`, `srcloc`, `where-constraints`, `implicit-binders`, `rest-type`.

3.2 Extended Spec Syntax

The extension uses Prologos's existing implicit map syntax. Keyword-headed children after the type signature are collected into a metadata map:

```
spec sort [List A] -> [List A]
  :implicits {A : Type}
  :where (Ord A)
  :doc "Sorts a list in ascending order using merge sort"
  :examples
    - [sort '[3N 1N 2N]] => '[1N 2N 3N]
    - [sort '[]] => '[]
  :properties (sortable-laws A)
  :see-also [reverse filter]
```

3.2.1 The `:implicits` Key

The `:implicits` key lifts implicit binders out of the type signature into metadata, freeing the signature to express pure function shape:

```
;; Current (inline implicits):
spec gmap {A B : Type} {C : Type -> Type} (Seqable C) -> (Buildable C) -> [A -> B] ->

;; With :implicits + :where:
spec gmap [A -> B] -> [C A] -> [C B]
  :implicits {A B : Type} {C : Type -> Type}
  :where (Seqable C) (Buildable C)
```

All three forms coexist:

1. Inline `{A : Type}` — for quick one-liners
2. `:implicits` key — for clean multi-line specs
3. Omitted entirely — when inference handles it (already works for kind `Type`)

Implementation: In `process-spec`, extract `:implicits` from the metadata map and merge with any inline `{...}` binders. The `extract-implicit-binders` function already returns `(values implicit-binders remaining-tokens)` — union the two sources. Duplicate binders (same name in both) emit a warning and deduplicate.

3.2.2 Multi-Arity Specs with Metadata

Decision: metadata is function-level (shared across all arities). Each `|` branch defines its own type; the metadata applies to the function as a whole.

```
spec greet
  | Nat -> String
  | String Nat -> String
  :doc "Greet someone"
  :examples
    - [greet 42N] => "hello 42"
    - [greet "world" 3N] => "hello world world world"
  :properties
    - :name "greet-nat-positive"
      :forall {n : Nat}
      :holds [str-length [greet n] > 0N]
```

Rationale: properties have `:forall` binders whose types determine which branch they exercise. Examples are concrete calls whose argument types determine dispatch. Per-branch metadata is theoretically more precise but practically redundant — the types already carry the dispatch information.

3.2.3 Sexp Mode Equivalent

```
(spec sort [List A] -> [List A]
  (:implicits {A : Type})
  (:where (Ord A))
  (:doc "Sorts a list in ascending order using merge sort")
  (:examples (([sort '[3N 1N 2N]] => [1N 2N 3N])
```

```
            ([sort '[]] => []))
  (:properties (sortable-laws A)))
```

3.2.4 How Parsing Works

1. The WS reader produces the outer form with keyword-headed children as nested s-expressions.

2. `rewrite-implicit-map` (macros.rkt line 2452) is extended to recognize `spec` as a trigger head alongside `def~/~defn`. The function detects keyword-headed children *after* the type signature and wraps them into a `$brace-params` (map literal) node.

3. `process-spec` (macros.rkt line 1314) is extended to look for a trailing map after extracting the type signature. Recognized keys are extracted; the entire map is stored in a new `metadata` field on `spec-entry`.

4. Backward compatibility: specs without keyword children parse identically to today. The existing positional `where` keyword still works; `:where` in the map augments it (union of both constraint sets).

3.3 Supported Keys

3.3.1 Phase 1 (Near-term: syntax + storage)

Key	Type	Semantics
`:implicits`	`(listof brace-param-group)`	Implicit type binders (moved from inline)
`:where`	`(listof constraint)`	Trait constraints (migrated from positional)
`:doc`	`string`	Documentation string
`:examples`	`(listof (datum => datum))`	Input/output pairs for auto-test generation
`:see-also`	`(listof symbol)`	Cross-references to related specs
`:since`	`string`	Version/date introduced
`:deprecated`	`string` or `#t`	Deprecation notice

3.3.2 Phase 2 (Medium-term: property checking)

Key	Type	Semantics
`:properties`	`(listof property-ref)`	Property references or inline property clauses
`:pre`	`datum` (args -> Bool)	Precondition (contract on arguments)
`:post`	`datum` (args ret -> Bool)	Postcondition (contract on return)
`:invariant`	`datum` ({args ret} -> Bool)	Relation between args and return (Clojure `:fn`)

3.3.3 Phase 3 (Long-term: verification)

Key	Type	Semantics
`:refines`	`datum` (result -> Bool)	Refinement predicate, compiles to Sigma type
`:measure`	`datum` (args -> Nat)	Termination measure for recursion
`:decreases`	`symbol` or `datum`	What decreases on recursive calls
`:proof`	`datum` or `:auto`	Proof strategy hint for verification

3.4 Mapping Spec Keys to Dependent Types

This is the deep theoretical connection between spec metadata and Prologos's type system. Every metadata key has a *type-theoretic interpretation*.

3.4.1 Properties as Dependent Types

A property "for all x : Nat, add x 0 = x" is a Pi type:

```
;; The property:
:properties
  - :name "right-identity"
    :forall {x : Nat}
    :holds [eq? [add x 0N] x]

;; Corresponds to the dependent type:
;; <(x : Nat) -> [Eq Nat [add x 0N] x]>
;; A function from any Nat to a proof of equality
```

3.4.2 Preconditions as Refined Arguments

```
;; The contract:
spec divide Int Int -> Int
  :pre (fn [x y] [not [eq? y 0]])

;; Maps to the dependent type:
;; <(x : Int) -> (y : Int) -> [Not [Eq Int y 0]] -> Int>
;; Third argument is a proof that y /= 0
```

3.4.3 Postconditions as Refined Return Types

```
;; The contract:
spec abs Int -> Int
```

```
  :post (fn [x result] [>= result 0])

;; Maps to:
;; <(x : Int) -> <(result : Int) * [GTE result 0]>>
;; Return type is a Sigma pair: the result AND a proof it's non-negative
```

3.4.4 Invariants as Dependent Function Types

```
;; Clojure spec's :fn relation:
spec ranged-rand Int Int -> Int
  :invariant (fn [start end result]
    [and [>= result start] [< result end]])

;; Maps to:
;; <(start : Int) -> (end : Int) -> {v : Int | v >= start, v < end}>
```

3.4.5 The Graduation Path

The key design insight: *the same spec syntax upgrades from testing to proving without modification.* At Phase 2, properties/contracts are runtime checks (QuickCheck generators from types, contract wrappers at call boundaries). At Phase 3, the same properties become proof obligations that the type checker verifies statically. The syntax is identical; the verification backend upgrades.

Phase	`:properties`	`:pre` / `:post`	`:refines`
2	Runtime QuickCheck	Runtime contracts	Runtime assertion
3	Proof obligation	Refined Pi type	Sigma compilation

4 Part III: Typed Holes (??)

4.1 Motivation

Agda and Idris demonstrate that *holes* are the foundation of interactive development in dependently-typed languages. The programmer writes what they know, marks what they don't with a hole, and the type system reports what's needed.

Prologos already has `_` (inference hole / wildcard), which means "infer this for me automatically." The `??` typed hole means something fundamentally different: "I don't know what goes here — show me what's possible."

4.2 Syntax

```
;; Unnamed hole:
defn reverse
  | [nil]        -> nil
  | [[cons h t]] -> ??

;; Named hole:
defn zip-with [f xs ys]
  match [xs ys]
    | [nil _]              -> nil
    | [_ nil]              -> nil
    | [[cons x xs'] [cons y ys']] -> [cons ??combine ??recurse]
```

4.2.1 Reader

The WS reader recognizes `??` (and `??identifier`) as a token type `'typed-hole`. The form-builder emits `($typed-hole)` or `($typed-hole name)` sentinel. The parser converts this to `expr-typed-hole` (new AST node).

4.2.2 Distinction from _

Feature	_ (inference hole)	?? (typed hole)
Intent	"Infer this for me"	"Help me fill this in"
Behavior	Silent, automatic	Interactive, diagnostic
Output	Resolved type/value	Hole report to editor
Use case	Type inference	Development workflow

4.2.3 Hole Report

When the type checker encounters `expr-typed-hole`, it produces a diagnostic (not an error) containing:

```
Hole ??combine : [List A]
Context:
  x  : A
  xs' : [List A]
  y  : B
  ys' : [List B]
  f  : A -> B -> C
  zip-with : [A -> B -> C] -> [List A] -> [List B] -> [List C]
Suggestions:
```

```
[f x y]             -- from applying f to available arguments
[cons [f x y] ...]  -- from matching the expected List type
```

The editor protocol (future work) would transmit this as structured data.

4.3 "The What?What?" Interaction Model

The envisioned workflow:

1. Write `spec` with type signature, properties, examples
2. Write `defn` skeleton with `??` holes for unknown parts
3. Press key command — the type system reports what each hole needs
4. Select a suggestion or refine manually
5. Repeat until all holes are filled and properties hold

This is fundamentally the Agda/Idris interaction model, but using Prologos's structured spec metadata to provide richer suggestions (e.g., if a property says the function is commutative, the case-split suggestions can leverage that).

5 Part IV: The `property` Keyword

5.1 Motivation

Many functions share the same properties. Sorting functions are idempotent. Monoids have identity and associativity. Functors preserve composition. Writing these properties inline on every `spec` is repetitive and error-prone.

The `property` keyword provides reusable, composable proposition groups — analogous to `bundle` for traits. The parallel is exact:

Concept	For methods	For propositions
Single declaration	`trait`	individual `:holds` clause
Named group	`bundle`	`property`
Composition	`bundle` includes traits/bundles	`property` includes properties
Attachment to spec	`:where (Ord A)`	`:properties (sortable-laws A)`
Attachment to trait	(implicit)	`:laws (functor-laws F)`

5.2 Syntax

5.2.1 Basic Declaration

```
property sortable-laws {A : Type}
  :where (Ord A)
  - :name "idempotent"
    :forall {xs : [List A]}
    :holds [eq? [sort [sort xs]] [sort xs]]
  - :name "length-preserving"
    :forall {xs : [List A]}
    :holds [eq? [length [sort xs]] [length xs]]
  - :name "ordered-output"
    :forall {xs : [List A]}
    :holds [sorted? [sort xs]]
```

A `property` is a named, parameterized conjunction of propositions. Each clause is a universally quantified boolean assertion. The `:where` constrains what types the property is meaningful for.

5.2.2 Usage in `spec`

```
spec sort [List A] -> [List A]
  :implicits {A : Type}
  :where (Ord A)
  :properties (sortable-laws A)
```

The `(sortable-laws A)` reference is like `(Ord A)` in a `:where` — a named requirement, parameterized by type variables from the spec.

5.3 Composition via `:includes`

Properties compose via conjunction, paralleling `bundle`:

```
property semigroup-laws {A : Type}
  :where (Add A)
  - :name "associativity"
    :forall {x y z : A}
    :holds [eq? [add [add x y] z] [add x [add y z]]]

property monoid-laws {A : Type}
  :where (Add A) (AdditiveIdentity A)
```

```
  :includes (semigroup-laws A)
  - :name "left-identity"
    :forall {x : A}
    :holds [eq? [add additive-identity x] x]
  - :name "right-identity"
    :forall {x : A}
    :holds [eq? [add x additive-identity] x]

property group-laws {A : Type}
  :where (Add A) (AdditiveIdentity A) (Neg A)
  :includes (monoid-laws A)
  - :name "left-inverse"
    :forall {x : A}
    :holds [eq? [add [neg x] x] additive-identity]
  - :name "right-inverse"
    :forall {x : A}
    :holds [eq? [add x [neg x]] additive-identity]
```

`group-laws` includes `monoid-laws` which includes `semigroup-laws`. Flattened: `group-laws` is the conjunction of all six properties.

5.4 The Logic Programming Reading

From the sequent calculus perspective:

```
property P {A} :where (C A) (C A) :includes (Q A) =
  , C(A), C(A)  Q(A)  p(A)  p(A)  ...
```

Each `:holds` clause is a proposition. `:includes` is conjunction. The whole `property` is a universally quantified sequent: "for all A satisfying C and C, the conjunction of all these propositions holds."

When attached to a spec with `:properties (P A)`, you assert that your implementation satisfies that sequent. At Phase 2 (QuickCheck), each conjunct becomes a test. At Phase 3 (verification), each conjunct becomes a proof obligation.

The composability is clean because conjunction is associative and commutative — the order and nesting of `:includes` don't matter; the result is always a flat set of propositions.

5.5 Parameterized Properties (Higher-Order)

Properties can be parameterized over functions, not just types:

```
;; Properties about ANY sorting function
property sorting-properties {A : Type} {f : [List A] -> [List A]}
  :where (Ord A)
  - :name "idempotent"
    :forall {xs : [List A]}
    :holds [eq? [f [f xs]] [f xs]]
  - :name "permutation"
    :forall {xs : [List A]}
    :holds [same-elements? xs [f xs]]
  - :name "ordered"
    :forall {xs : [List A]}
    :holds [sorted? [f xs]]

;; Attach to multiple implementations:
spec sort [List A] -> [List A]
  :implicits {A : Type}
  :where (Ord A)
  :properties (sorting-properties A sort)

spec merge-sort [List A] -> [List A]
  :implicits {A : Type}
  :where (Ord A)
  :properties (sorting-properties A merge-sort)
```

Define the contract once, attach it to any implementation.

5.6 `:laws` on Traits

A trait can reference its laws:

```
property functor-laws {F : Type -> Type}
  :where (Functor F)
  - :name "identity"
    :forall {xs : [F A]}
    :holds [eq? [fmap id xs] xs]
  - :name "composition"
    :forall {f : [A -> B]} {g : [B -> C]} {xs : [F A]}
    :holds [eq? [fmap [>> f g] xs] [fmap g [fmap f xs]]]

trait Functor {F : Type -> Type}
```

```
  :laws (functor-laws F)
  spec fmap {A B : Type} [A -> B] -> [F A] -> [F B]
```

`:laws` says "any instance of this trait is expected to satisfy these properties." When you write `instance Functor List`, the system knows to check `(functor-laws List)`. At Phase 2: generate `List`-specialized QuickCheck tests. At Phase 3: proof obligations.

5.7 Hierarchical Laws (Functor → Applicative → Monad)

```
property applicative-laws {F : Type -> Type}
  :where (Applicative F)
  :includes (functor-laws F)
  - :name "identity"
    :forall {xs : [F A]}
    :holds [eq? [ap [pure id] xs] xs]
  - :name "homomorphism"
    :forall {f : [A -> B]} {x : A}
    :holds [eq? [ap [pure f] [pure x]] [pure [f x]]]

property monad-laws {M : Type -> Type}
  :where (Monad M)
  :includes (applicative-laws M)
  - :name "left-identity"
    :forall {a : A} {f : [A -> [M B]]}
    :holds [eq? [bind [pure a] f] [f a]]
  - :name "right-identity"
    :forall {ma : [M A]}
    :holds [eq? [bind ma pure] ma]
  - :name "associativity"
    :forall {ma : [M A]} {f : [A -> [M B]]} {g : [B -> [M C]]}
    :holds [eq? [bind [bind ma f] g] [bind ma [fn [a] [bind [f a] g]]]]
```

5.8 Name Scoping in `:includes`

When `monad-laws` includes `applicative-laws` which includes `functor-laws`, and both `monad-laws` and `functor-laws` define a property named `"identity"`: names are scoped to their declaring `property` block.

Flattened property names use `/` qualification: `functor-laws/identity`, `applicative-laws/identity`, `monad-laws/left-identity`.

This parallels how trait methods are scoped to their trait.

5.9 The Symmetry

```
trait     : Type → Set of method signatures     (what you must implement)
bundle    : conjunction of traits                (shorthand for multiple traits)
property  : Type → Set of propositions           (what must be true)
  includes: conjunction of properties            (shorthand for multiple properties)

spec      : function → type + metadata
  :where  : trait constraints                    (what the types must support)
  :properties : property constraints             (what the function must satisfy)

instance  : Type → method implementations        (how you implement)
  (auto)  : Type → property evidence             (how you prove / test)
```

Traits say what operations exist. Properties say what laws they obey. Today only the first is checked; this design gives us a path to checking the second.

6 Part V: Expressing Higher-Kinded Pi Types Ergonomically

6.1 The Problem

Complex higher-kinded and higher-rank types are unreadable:

```
;; Current xf-compose signature:
spec xf-compose {A B C : Type} <(S :0 Type) -> [S -> B -> S] -> S -> A -> S> <(S :0 Typ
```

The transducer module already identifies the abstraction in a comment: `Type: Xf A B = forall R. (R -> B -> R) -> (R -> A -> R)`. With the right name, the signature becomes: `[Xf A B] -> [Xf B C] -> [Xf A C]` — category-theoretic composition, instantly readable.

6.2 Progressive Complexity Levels

6.2.1 Level 0: No types (inference only)

```
defn add [x y] [nat-add x y]
```

6.2.2 Level 1: Simple spec (monomorphic)

```
spec add Nat Nat -> Nat
defn add [x y] [nat-add x y]
```

6.2.3 Level 2: Polymorphic spec

```
spec id A -> A
defn id [x] x
```

(`{A : Type}` inferred from `A` appearing free in signature)

6.2.4 Level 3: Constrained polymorphism

```
spec sort [List A] -> [List A]
  :where (Ord A)
defn sort [xs] ...
```

6.2.5 Level 4: Higher-kinded polymorphism

```
spec fmap [A -> B] -> [F A] -> [F B]
  :where (Functor F)
  :doc "Apply a function inside a functor"
  :properties (functor-laws F)
defn fmap [f xs] ...
```

(`{A B : Type}` inferred; `{F : Type -> Type}` inferred from `(Functor F)` where clause — the trait declares the kind)

6.2.6 Level 5: Named type abstractions (functor)

```
spec xf-compose [Xf A B] -> [Xf B C] -> [Xf A C]
  :implicits {A B C : Type}
  :properties (transducer-fusion-laws A B C)
```

(`Xf` defined via `functor` — see Part VII)

6.2.7 Level 6: Dependent types (Pi)

```
spec replicate <(n : Nat) -> A -> [Vec A n]>
  :implicits {A : Type}
  :doc "Create a vector of n copies of a value"
  :examples
    - [replicate 3N :x] => @[:x :x :x]
  :properties
    - :name "correct-length"
      :forall {n : Nat} {a : A}
      :holds [eq? [length [replicate n a]] n]
```

6.2.8 Level 7: Dependent types (Sigma / existential)

```
spec filter [A -> Bool] -> [List A] -> <(result : [List A]) * [SubList result xs]>
  :implicits {A : Type}
  :doc "Filter elements, with proof that result is a sublist"
```

6.3 Key Ergonomic Decisions

1. **Pi/Sigma types never need to leak to the surface**: `functor` declarations name complex type abstractions with approachable metadata. The underlying Pi type lives in `:unfolds`, consulted only when needed.

2. **Angle brackets contain the complexity**: `<...>` delimits the dependent part of a type. Everything outside angle brackets is familiar ML-style syntax.

3. **Metadata explains the theory**: The `:doc` key provides human-readable explanation. Properties double as documentation and specification.

4. **Examples ground the abstraction**: Every complex type should have concrete examples. The `:examples` key makes this a first-class concern.

5. **Where-clauses read as English**: "sort takes a list of A where A has ordering" is near-English. The trait constraint system hides the dictionary-passing mechanism.

6. **Properties express what the function *means***: Rather than encoding everything in the type (which can be opaque), properties express semantic invariants in a universally readable format.

7. `:implicits` **declutters signatures**: Implicit binders move to metadata when they would add visual weight to the type signature.

7 Part VI: Improved Implicit Inference

Today, `{A : Type}` is required even when `A` appears in the type signature (e.g., `[List A] -> Nat`). The type checker can already infer this in many cases. Two improvements are proposed:

7.1 Direction 1: Auto-introduce unbound type variables

If a capitalized identifier `A` appears free in the type signature and is not bound by any explicit binder, auto-introduce `{A : Type}`. This already works for simple cases; the proposal is to make it the documented, reliable behavior.

```
;; These would be equivalent:
spec length {A : Type} [List A] -> Nat
spec length [List A] -> Nat
```

7.2 Direction 2: Kind inference from `:where` clauses

If `C` appears in `:where (Seqable C)` and `Seqable` is declared over `{C : Type -> Type}`, infer `C`'s kind from the where clause. This eliminates the most annoying case — having to write `{C : Type -> Type}` just because you use an HKT trait.

```
;; These would be equivalent:
spec gmap {A B : Type} {C : Type -> Type} [A -> B] -> [C A] -> [C B]
  :where (Seqable C) (Buildable C)

spec gmap [A -> B] -> [C A] -> [C B]
  :where (Seqable C) (Buildable C)
```

Note: `propagate-kinds-from-constraints` already does this internally. The proposal is to make it work when explicit `{...}` binders are omitted entirely.

7.3 Explicit Always Available

Explicit binders remain for: pedagogic clarity, disambiguation when inference produces unexpected results, and the rare cases where no constraining position exists (e.g., `spec empty {A : Type} [List A]`).

8 Part VII: The `functor` Keyword

8.1 Motivation

The `functor` keyword provides named type abstractions with category-theoretic metadata. It solves the fundamental readability problem of higher-kinded and higher-rank types by giving them *meaning through structure*, not just through their expansion.

The design philosophy: you understand `Xf A B` by knowing what it *does* (":doc A transducer"), how it *combines* (":compose xf-compose"), and what *rules* it follows (":laws transducer-fusion-laws"). The Pi type it unfolds to is available but is consulted last, not first.

8.2 Syntax

```
functor Xf {A B : Type}
  :doc "A transducer: transforms A-reductions into B-reductions"
  :compose xf-compose
  :identity id-xf
  :laws (transducer-fusion-laws A B)
  :unfolds <(S :0 Type) -> [S -> B -> S] -> S -> A -> S>
```

8.3 Progressive Disclosure

Like `spec`, the metadata keys are progressive. Simple synonyms are minimal:

```
functor FilePath
  :unfolds String
```

Parameterized synonyms add type parameters:

```
functor Result {A}
  :unfolds [Either String A]
  :doc "A computation that may fail with a string error"
```

Algebraically structured types add operations:

```
functor Xf {A B : Type}
  :doc "A transducer: transforms A-reductions into B-reductions"
  :compose xf-compose
  :identity id-xf
  :unfolds <(S :0 Type) -> [S -> B -> S] -> S -> A -> S>
```

Full category-theoretic treatment adds laws:

```
functor Lens {S T A B : Type}
  :doc "A bidirectional accessor: view and update a part of a structure"
  :compose lens-compose
  :identity lens-id
  :laws (lens-laws S T A B)
  :see-also [Prism Traversal Iso]
  :unfolds <{F : Type -> Type} -> (Functor F) -> [A -> [F B]] -> S -> [F T]>
```

8.4 Category-Theoretic Metadata Keys

The keys map CT concepts to approachable language:

CT concept	Key	Meaning in plain English
Object mapping	`:unfolds`	"What this actually expands to under the hood"
Morphism composition	`:compose`	"How to chain two of these together"
Identity morphism	`:identity`	"The do-nothing version"
Laws	`:laws`	"What rules these always follow"
Documentation	`:doc`	"What this means in human terms"
Natural transformation	`:transforms`	"How to convert between different functors" (future)
Adjunction	`:adjoint`	"What functor is this paired with" (far future)

Only `:unfolds` is required (for Phase 1). The rest are progressive. A user who never thinks about category theory writes:

```
functor Xf {A B : Type}
  :doc "A transducer"
  :unfolds <(S :0 Type) -> [S -> B -> S] -> S -> A -> S>
```

A user who cares about algebraic structure adds `:compose` and `:identity`. A user who wants verification adds `:laws`. The CT is always there but only surfaces when you reach for it.

8.5 Why `functor` and Not `abbrev` or `type`

`abbrev` doesn't capture the ambition. An abbreviation is just a shorter name for the same thing. What we're proposing is a *named abstraction with declared structure* — the operations, laws, documentation are first-class. `abbrev` makes you think of `typedef` — mechanical shortening. `functor` makes you think of structure-preserving mappings — which is what these actually are.

`type` clashes with `deftype` (which creates algebraic data types with constructors). A `functor` doesn't create constructors; it names a type with structure.

The `functor` (keyword) vs `Functor` (trait) naming is consistent with Prologos's existing pattern: all keywords are lowercase (`spec`, `defn`, `trait`, `bundle`); all type/trait names are capitalized. The relationship: a `functor` declaration might *itself* be a `Functor` instance (in the trait sense). The keyword names the concept; the trait constrains it.

8.6 Transparency Model

`functor` declarations are transparent by default (like Lean's `abbrev`). The type checker unfolds `Xf A B` to its `:unfolds` form during elaboration. Trait instances carry through: if `Eq (Xf Nat Bool)` is needed and `Eq` has an instance for the expanded type, it resolves.

Future consideration: opaque functors (no `:unfolds`, only described by operations) for abstract type boundaries. Deferred to post-Phase 1.

8.7 Error Messages

Error messages show the `functor` name by default:

```
Error: Expected [Xf Nat Bool], got [Xf Nat Nat]
```

Not the expanded form. An `--expand-types` flag or editor hover reveals the full type when needed.

8.8 How `functor` Composes with `spec`, `property`, and `trait`

```
;; The type abstraction
functor Xf {A B : Type}
  :doc "A transducer: transforms A-reductions into B-reductions"
  :compose xf-compose
  :identity id-xf
  :laws (transducer-fusion-laws A B)
  :unfolds <(S :0 Type) -> [S -> B -> S] -> S -> A -> S>

;; Reusable properties (using the functor name!)
property transducer-fusion-laws {A B C : Type}
  - :name "compose-associativity"
    :forall {f : [Xf A B]} {g : [Xf B C]} {h : [Xf C D]}
    :holds [eq? [xf-compose [xf-compose f g] h] [xf-compose f [xf-compose g h]]]
  - :name "left-identity"
    :forall {f : [Xf A B]}
    :holds [eq? [xf-compose id-xf f] f]
  - :name "right-identity"
    :forall {f : [Xf A B]}
    :holds [eq? [xf-compose f id-xf] f]

;; Function specs using the functor name
```

```
spec xf-compose [Xf A B] -> [Xf B C] -> [Xf A C]
  :implicits {A B C : Type}
  :doc "Compose two transducers (apply first argument first)"
  :properties (transducer-fusion-laws A B C)

spec map-xf [A -> B] -> [Xf A B]
  :implicits {A B : Type}
  :doc "Transform each element through f before passing to reducer"

spec filter-xf [A -> Bool] -> [Xf A A]
  :implicits {A : Type}
  :doc "Only pass elements satisfying pred to the reducer"

spec transduce [Xf A B] -> [R -> B -> R] -> R -> [List A] -> R
  :implicits {A B R : Type}
  :doc "Apply a transducer to a list with a reducer and initial accumulator"

spec into-list [Xf A B] -> [List A] -> [List B]
  :implicits {A B : Type}
  :doc "Transduce into a correctly-ordered list"
  :examples
    - [into-list [map-xf suc] '[1N 2N 3N]] => '[2N 3N 4N]
```

The `functor` names the type. The `property` states its laws. The `spec` uses both. Everything is readable. The Pi types are present exactly once, in `:unfolds`, consulted only when necessary.

9 Part VIII: Implementation Considerations

9.1 spec-entry Struct Extension

The recommended approach: add a single `metadata` field that stores the entire keyword map as a hash table. Individual accessors extract from the metadata hash. This avoids struct migration pain:

```
(struct spec-entry
  (type-datums docstring multi? srcloc where-constraints
   implicit-binders rest-type
   metadata)  ;; <-- one new field: hasheq of keyword -> datum
  #:transparent)
```

```
;; Accessor helpers:
(define (spec-entry-examples e)
  (hash-ref (or (spec-entry-metadata e) (hasheq)) ':examples '()))
(define (spec-entry-properties e)
  (hash-ref (or (spec-entry-metadata e) (hasheq)) ':properties '()))
(define (spec-entry-implicits-meta e)
  (hash-ref (or (spec-entry-metadata e) (hasheq)) ':implicits #f))
```

9.2 rewrite-implicit-map Extension

Currently scoped to `def` and `defn` heads (macros.rkt line 2452). Extending to `spec` is a one-line change in the `memq` check. However, the spec head requires special handling: the type signature tokens must be separated from the trailing keyword map. This can be done by scanning for the first keyword-headed child whose indentation matches the spec's indentation level.

9.3 Typed Hole AST Node

```
(struct expr-typed-hole (name) #:transparent)  ;; name is #f or symbol
```

Type checker (typing-core.rkt): when encountering `expr-typed-hole`, compute the expected type from the context and emit a diagnostic report rather than returning `(expr-error)`.

9.4 property-entry Store

```
(struct property-entry
  (name           ;; symbol
   params         ;; alist of ((name . kind) ...)
   where-clauses  ;; (listof constraint)
   includes       ;; (listof (property-ref ...))
   clauses        ;; (listof property-clause)
   metadata)      ;; hasheq for extensibility
  #:transparent)

(struct property-clause
  (name           ;; string (human-readable label)
   forall-binders ;; alist of ((name . type) ...)
   holds-expr)    ;; datum (boolean expression)
  #:transparent)
```

```
(define current-property-store (make-parameter (hasheq)))
```

`:includes` references are resolved at registration time by looking up the included property in the store and appending its (flattened) clauses. Clause names are prefixed with the declaring property's name and `/`.

9.5 functor-entry Store

```
(struct functor-entry
  (name         ;; symbol (capitalized)
   params       ;; alist of ((name . kind) ...)
   unfolds      ;; type datum (the expansion)
   metadata)    ;; hasheq (:compose, :identity, :laws, :doc, etc.)
  #:transparent)

(define current-functor-store (make-parameter (hasheq)))
```

During elaboration, when the elaborator encounters a capitalized identifier followed by arguments that matches a registered functor, it expands to the `unfolds` form with parameters substituted.

9.6 `:implicits` Merging in process-spec

```
;; In process-spec, after extracting metadata:
(define meta-implicits
  (if metadata
      (extract-binders-from-implicits-key metadata)
      '()))
(define merged-implicits
  (deduplicate-binders (append inline-implicits meta-implicits)))
;; Warn on duplicates between inline and :implicits
```

9.7 Grammar Extensions

Additions to `grammar.ebnf`:

```
(* Typed holes for interactive development *)
typed-hole      = '??' , [ identifier ] ;

(* Extended spec with optional metadata *)
```

```
spec-form       = 'spec' , identifier , spec-body , [ spec-metadata ] ;
spec-metadata   = spec-meta-entry , { spec-meta-entry } ;
spec-meta-entry = keyword-literal , spec-meta-value ;
spec-meta-value = expr
                | dash-list
                | spec-meta-map
                ;

(* Property declarations *)
property-form   = 'property' , identifier , [ implicit-binders ]
                , [ property-metadata ] , { property-clause } ;
property-metadata = property-meta-entry , { property-meta-entry } ;
property-meta-entry = ':where' , '(' , constraint , { constraint } , ')'
                    | ':includes' , '(' , property-ref , { property-ref } , ')'
                    ;
property-clause = '-' , ':name' , string-literal
                , ':forall' , '{' , binder , { binder } , '}'
                , ':holds' , expr ;

(* Functor declarations *)
functor-form    = 'functor' , identifier , [ implicit-binders ]
                , functor-metadata ;
functor-metadata = functor-meta-entry , { functor-meta-entry } ;
functor-meta-entry = ':unfolds' , type-expr
                   | ':compose' , identifier
                   | ':identity' , identifier
                   | ':laws' , '(' , property-ref , { property-ref } , ')'
                   | ':doc' , string-literal
                   | ':see-also' , '[' , identifier , { identifier } , ']'
                   ;
```

10 Part IX: Worked Examples

10.1 Example 1: Simple spec with documentation, examples, and reusable properties

```
property cons-head-law {A : Type}
  - :name "head-of-cons"
    :forall {x : A} {xs : [List A]}
    :holds [eq? [head [cons x xs]] [just x]]
```

```
spec head [List A] -> A?
  :implicits {A : Type}
  :doc "Returns the first element, or nothing if empty"
  :examples
    - [head '[1N 2N 3N]] => [just 1N]
    - [head '[]] => nothing
  :properties (cons-head-law A)

defn head
  | [nil]        -> nothing
  | [[cons h _]] -> [just h]
```

10.2 Example 2: Contract-style spec

```
spec divide Int Int -> Int
  :pre (fn [_ y] [not [eq? y 0]])
  :post (fn [x y r] [eq? [+ [* r y] [int-mod x y]] x])
  :doc "Integer division; undefined for zero divisor"

defn divide [x y] [int-div x y]
```

10.3 Example 3: Higher-kinded functor laws (using property + :laws)

```
property functor-laws {F : Type -> Type}
  :where (Functor F)
  - :name "identity-law"
    :forall {xs : [F A]}
    :holds [eq? [fmap id xs] xs]
  - :name "composition-law"
    :forall {f : [A -> B]} {g : [B -> C]} {xs : [F A]}
    :holds [eq? [fmap [>> f g] xs] [fmap g [fmap f xs]]]

trait Functor {F : Type -> Type}
  :laws (functor-laws F)
  spec fmap {A B : Type} [A -> B] -> [F A] -> [F B]

spec fmap [A -> B] -> [F A] -> [F B]
  :where (Functor F)
```

```
  :properties (functor-laws F)
```

10.4 Example 4: Transducer module (functor + property + spec)

```
ns prologos::data::transducer

functor Xf {A B : Type}
  :doc "A transducer: transforms A-reductions into B-reductions"
  :compose xf-compose
  :identity id-xf
  :laws (transducer-fusion-laws A B)
  :unfolds <(S :0 Type) -> [S -> B -> S] -> S -> A -> S>

property transducer-fusion-laws {A B C : Type}
  - :name "compose-associativity"
    :forall {f : [Xf A B]} {g : [Xf B C]} {h : [Xf C D]}
    :holds [eq? [xf-compose [xf-compose f g] h] [xf-compose f [xf-compose g h]]]
  - :name "left-identity"
    :forall {f : [Xf A B]}
    :holds [eq? [xf-compose id-xf f] f]
  - :name "right-identity"
    :forall {f : [Xf A B]}
    :holds [eq? [xf-compose f id-xf] f]

spec map-xf [A -> B] -> [Xf A B]
  :implicits {A B : Type}
  :doc "Transform each element through f before passing to reducer"

spec filter-xf [A -> Bool] -> [Xf A A]
  :implicits {A : Type}
  :doc "Only pass elements satisfying pred to the reducer"

spec xf-compose [Xf A B] -> [Xf B C] -> [Xf A C]
  :implicits {A B C : Type}
  :doc "Compose two transducers (apply first argument first)"
  :properties (transducer-fusion-laws A B C)

spec transduce [Xf A B] -> [R -> B -> R] -> R -> [List A] -> R
  :implicits {A B R : Type}
```

```
  :doc "Apply a transducer to a list with a reducer and initial accumulator"

spec into-list [Xf A B] -> [List A] -> [List B]
  :implicits {A B : Type}
  :doc "Transduce into a correctly-ordered list"
  :examples
    - [into-list [map-xf suc] '[1N 2N 3N]] => '[2N 3N 4N]
```

10.5 Example 5: Interactive development with ??

```
spec reverse [List A] -> [List A]
  :implicits {A : Type}
  :properties
    - :name "involution"
      :forall {xs : [List Nat]}
      :holds [eq? [reverse [reverse xs]] xs]
    - :name "length-preserving"
      :forall {xs : [List Nat]}
      :holds [eq? [length [reverse xs]] [length xs]]

defn reverse
  | [nil]        -> nil
  | [[cons h t]] -> ??
  ;; Type system reports:
  ;;   Hole ?? : [List A]
  ;;   Context: h : A, t : [List A], reverse : [List A] -> [List A]
  ;;   Suggestions: [append [reverse t] [cons h nil]]
```

10.6 Example 6: Algebraic property hierarchy

```
property semigroup-laws {A : Type}
  :where (Add A)
  - :name "associativity"
    :forall {x y z : A}
    :holds [eq? [add [add x y] z] [add x [add y z]]]

property monoid-laws {A : Type}
  :where (Add A) (AdditiveIdentity A)
  :includes (semigroup-laws A)
  - :name "left-identity"
```

```
    :forall {x : A}
    :holds [eq? [add additive-identity x] x]
  - :name "right-identity"
    :forall {x : A}
    :holds [eq? [add x additive-identity] x]

;; Usage:
spec fold [A -> A -> A] -> A -> [List A] -> A
  :implicits {A : Type}
  :where (Add A) (AdditiveIdentity A)
  :properties (monoid-laws A)
```

10.7 Example 7: Optics (functor for Lens)

```
functor Lens {S T A B : Type}
  :doc "A bidirectional accessor: view and update a part of a structure"
  :compose lens-compose
  :identity lens-id
  :laws (lens-laws S T A B)
  :see-also [Prism Traversal Iso]
  :unfolds <{F : Type -> Type} -> (Functor F) -> [A -> [F B]] -> S -> [F T]>

spec lens-view [Lens S S A A] -> S -> A
  :doc "Extract the focus from a structure"

spec lens-set [Lens S S A A] -> A -> S -> S
  :doc "Replace the focus in a structure"
  :properties
    - :name "set-get"
      :forall {l : [Lens S S A A]} {s : S} {a : A}
      :holds [eq? [lens-view l [lens-set l a s]] a]
```

10.8 Example 8: Refinement type (future)

```
spec abs Int -> Int
  :refines (fn [r] [>= r 0])
  :doc "Absolute value --- result is always non-negative"

;; Internally, :refines compiles to return type:
;; <(r : Int) * [GTE r 0]>
```

```
;; The function returns a pair: the result AND a proof of non-negativity
```

10.9 Example 9: Dependent-type-heavy spec

```
spec zip-with-vec <(n : Nat) -> [A -> B -> C] -> [Vec A n] -> [Vec B n] -> [Vec C n]>
  :implicits {A B C : Type}
  :doc "Zip two vectors of the same length with a combining function"
  :examples
    - [zip-with-vec 3N + @[1 2 3] @[10 20 30]] => @[11 22 33]
  :properties
    - :name "preserves-length"
      :forall {n : Nat} {f : [Int -> Int -> Int]}
              {xs : [Vec Int n]} {ys : [Vec Int n]}
      :holds [eq? [vec-length [zip-with-vec n f xs ys]] n]
```

11 Part X: Phased Roadmap

11.1 Phase 1: Syntax and Storage (Near-term)

In scope now:

- Extend `rewrite-implicit-map` to trigger on `spec` heads
- Extend `process-spec` to extract keyword entries into metadata hash
- Add `metadata` field to `spec-entry`
- `:implicits` key: extract from metadata, merge with inline binders
- `:doc` replaces positional docstring; `:where` augments positional `where`
- Store `:examples`, `:see-also` in metadata
- `??` reader token and `expr-typed-hole` AST node (report-only)
- `property` keyword: parser, store, `:includes` flattening, name-scoped clauses
- `:laws` key on `trait` declarations
- `functor` keyword: parser, store, transparent expansion during elaboration
- Grammar updates (grammar.ebnf, grammar.org)

11.2 Phase 1b: Improved Implicit Inference

- Auto-introduce unbound type variables as `{X : Type}`
- Kind inference from `:where` clauses (extend `propagate-kinds-from-constraints` to work when no explicit binders are present)
- Goal: `{A : Type}` almost never needed for kind-`Type` variables; `{C : Type -> Type}` unnecessary when `:where` pins the kind

11.3 Phase 2: Example and Property Checking (Medium-term)

- Parse `:examples` entries, type-check them, run them as tests
- `Gen` trait for type-directed random value generation
- Parse `:properties`, generate random inputs from types, check `:holds`
- Property checking for `:laws` on trait instances
- Contract wrapping: `:pre~`/:post~ generate runtime checks at function boundaries (Racket-contract style blame tracking)

11.4 Phase 3: Refinement and Verification (Long-term)

- `:refines` compiles to Sigma types (return type is `<(r : T) * P(r)>`)
- `:properties` become compile-time proof obligations
- Proof search integration: `:proof :auto` triggers the logic engine
- `:measure` for termination checking
- Opaque functors (no `:unfolds`) for abstract type boundaries

11.5 Phase 4: Interactive Theorem Proving (Far future)

- Editor protocol for `??` hole interaction
- Case splitting via type information (Agda `C-c C-c`)
- Proof search (Agda `C-c C-a`) via propagator-based logic engine
- Refinement reflection (Liquid Haskell style)
- `:transforms` and `:adjoint` keys on `functor` for natural transformations

12 Part XI: Resolved Design Questions

1. **Should `:where` in the map replace or augment positional `where`? Decision**: augment (union of both constraint sets). Positional form remains for backward compatibility; map form is preferred going forward.

2. **Should examples use `=>` or `->` as the separator? Decision**: `=>` to avoid confusion with the arrow in type signatures.

3. **How should multi-arity specs handle metadata? Decision**: metadata is function-level (shared across all arities). Each `|` branch defines its own type; the metadata applies to the function as a whole. Properties' `:forall` types determine which branch is exercised. Examples' argument types determine dispatch.

4. **Should `:properties` quantified variables shadow spec's implicit binders? Decision**: yes, with a warning. The property is a self-contained expression; it should be readable without consulting the outer spec.

5. **What happens to unrecognized keys? Decision**: store silently (forward-compatibility). Tool-specific keys (e.g., `:editor-hint`, `:benchmark`) can be added without language changes.

6. **Is `:unfolds` always required on `functor`? Decision**: yes, for Phase 1. Opaque functors (no `:unfolds`) deferred to Phase 3.

7. **Should `functor` handle non-CT uses (plain type synonyms)? Decision**: yes — `functor` is the single keyword for "I'm naming a type concept." Progressive metadata: no `:compose` means no algebraic structure.

13 References

13.1 Clojure Spec

- Clojure Spec Guide
- s/fdef Reference
- Spec Rationale and Overview

13.2 Malli

- Malli GitHub
- Malli Function Schemas

13.3 PropEr

- PropEr GitHub
- PropEr Tutorial

13.4 Agda

- Agda Auto Proof Search
- A Taste of Agda

13.5 Idris 2

- Idris 2 Interactive Editing

13.6 Lean 4

- Lean 4 Type Classes
- Lean 4 Instance Synthesis

13.7 Liquid Haskell

- Refinement Types Tutorial
- Refinement Types for Haskell (paper)

13.8 Haskell Type Synonyms

- GHC Type Synonyms

13.9 Koka

- Koka Language Book

13.10 F*

- F* Qualifier Wiki

13.11 1ML

- 1ML (Rossberg) — Core and Modules United

13.12 Racket Contracts

- Racket Contracts Guide